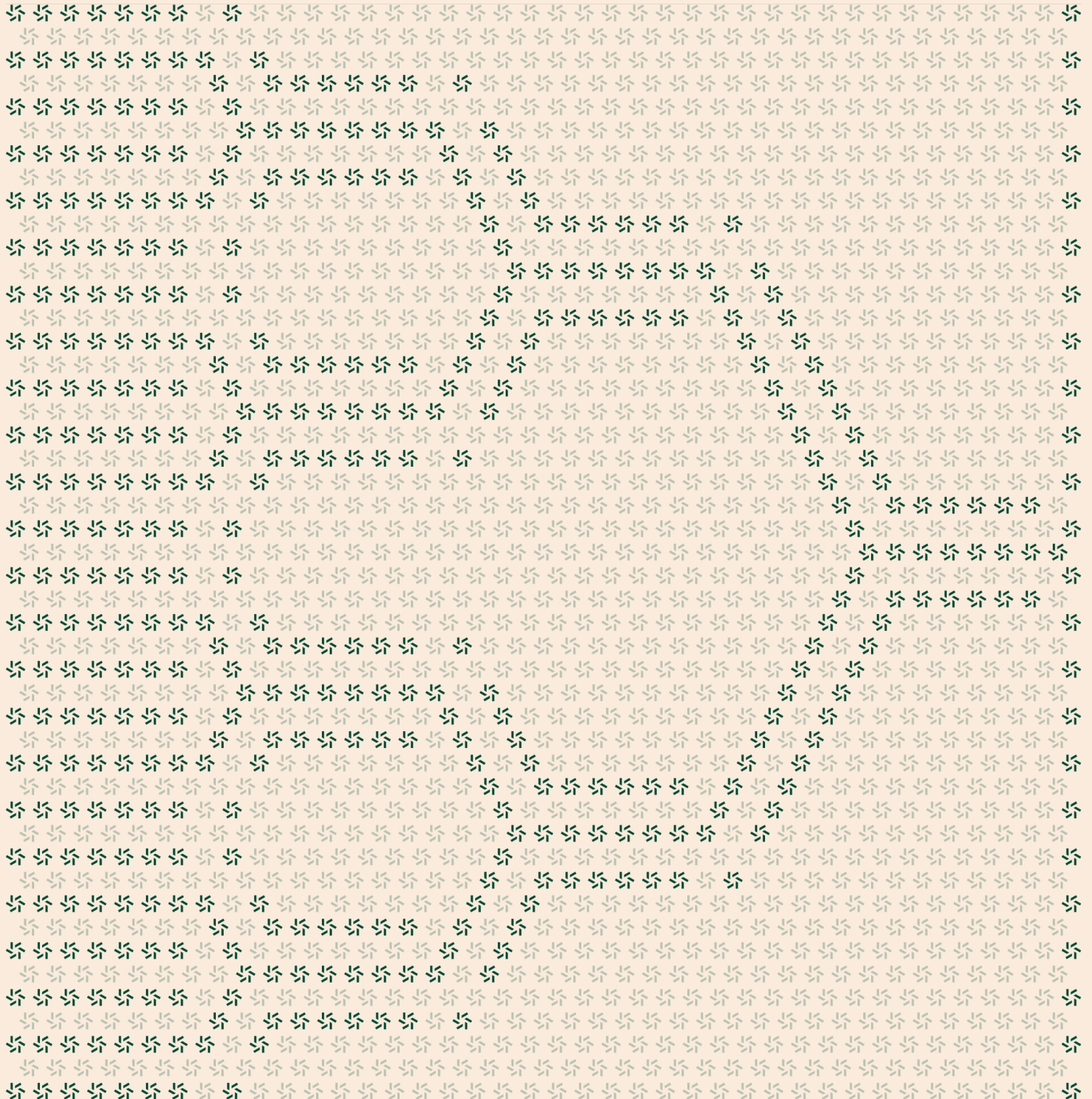


September 28, 2026

AltDA OP Succinct Integration

Zero Knowledge Application Security Assessment



Contents

About Zellic	4
1. Overview	5
1.1. Executive Summary	5
1.2. Goals of the Assessment	5
1.3. Non-goals and Limitations	5
1.4. Results	6
2. Introduction	7
2.1. About AltDA OP Succinct Integration	7
2.2. Methodology	7
2.3. Scope	9
2.4. Project Overview	9
2.5. Project Timeline	10
3. Detailed Findings	11
3.1. The zkVM does not bind the first digest byte of an AltDA Keccak commitment	11
3.2. The client applies the deposit-only fallback to every execution error	13
3.3. The AltDA host does not limit HTTP response bodies	15
4. System Design	17
4.1. Component: AltDA Client and Host	17

5.	Assessment Results	19
5.1.	Disclaimer	19

About Zellic

Zellic is a vulnerability research firm with deep expertise in blockchain security. We specialize in EVM, Move (Aptos and Sui), and Solana as well as Cairo, NEAR, and Cosmos. We review L1s and L2s, cross-chain protocols, wallets and applied cryptography, zero-knowledge circuits, web applications, and more.

Prior to Zellic, we founded the [#1 CTF \(competitive hacking\) team](#) [worldwide](#) in 2020, 2021, and 2023. Our engineers bring a rich set of skills and backgrounds, including cryptography, web security, mobile security, low-level exploitation, and finance. Our background in traditional information security and competitive hacking has enabled us to consistently discover hidden vulnerabilities and develop novel security research, earning us the reputation as the go-to security firm for teams whose rate of innovation outpaces the existing security landscape.

For more on Zellic's ongoing security research initiatives, check out our website zellic.io [and](#) follow [@zellic_io](#) [on Twitter](#). If you are interested in partnering with Zellic, contact us at hello@zellic.io [.](#)



1. Overview

1.1. Executive Summary

Zellic conducted a security assessment for Succinct Labs from September 18th to September 23rd, 2026. During this engagement, Zellic reviewed AltDA OP Succinct Integration's code for security vulnerabilities, design issues, and general weaknesses in security posture.

1.2. Goals of the Assessment

In a security assessment, goals are framed in terms of questions that we wish to answer. These questions are agreed upon through close communication between Zellic and the client. In this assessment, we sought to answer the following questions:

- Can a malicious host or DA server cause the guest to accept batch data that does not match the L1 commitment?
 - Are the starting state, L1 context, ending L2 block, and roll-up configuration correctly bound to the proof's public values?
 - Does AltDA input-size handling match the intended op-node behavior when `da_max_input_size` is omitted, zero, exactly at the limit, or exceeded? Is the configured limit included consistently in the roll-up configuration hash?
 - Do invalid or unsupported commitments, temporarily missing preimages, and pipeline resets produce the correct skip/retry behavior without losing or reusing input?
 - Can malformed witnesses or oversized DA responses cause excessive resource use before validation, including during host HTTP response handling?
-

1.3. Non-goals and Limitations

We did not assess the following areas that were outside the scope of this engagement:

- Front-end components
- Infrastructure relating to the project
- Key custody

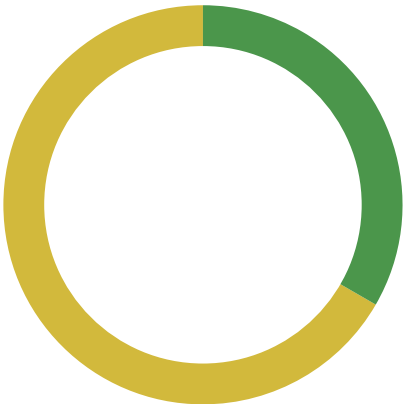
Due to the time-boxed nature of security assessments in general, there are limitations in the coverage an assessment can provide.

1.4. Results

During our assessment on the scoped AltDA OP Succinct Integration circuits, we discovered three findings. No critical issues were found. Two findings were of medium impact and one was of low impact.

Breakdown of Finding Impacts

Impact Level	Count
Critical	0
High	0
Medium	2
Low	1
Informational	0



2. Introduction

2.1. About AltDA OP Succinct Integration

Succinct Labs contributed the following description of AltDA OP Succinct Integration:

OP Succinct uses SP1 to prove OP Stack state transitions. This review covers the generic AltDA integration, which retrieves off-chain batch data referenced by Keccak256 commitments on L1 and validates that data before rollup derivation and execution.

2.2. Methodology

During a security assessment, Zellic works through standard phases of security auditing, including both automated testing and manual review. These processes can vary significantly per engagement, but the majority of the time is spent on a thorough manual review of the entire scope.

Alongside a variety of tools and analyzers used on an as-needed basis, Zellic focuses primarily on the following classes of security and reliability issues:

Underconstrained circuits. The most common type of vulnerability in a ZKP circuit is not adding sufficient constraints to the system. This leads to proofs generated with incorrect witnesses in terms of the specification of the project being accepted by the ZKP verifier. We manually check that the set of constraints satisfies soundness, enough to remove all such possibilities and in some cases provide a proof of the fact.

Overconstrained circuit. While rare, it is possible that a circuit is overconstrained. In this case, appropriately assigning witnesses will become impossible, leading to a vulnerability. To prevent this, we manually check that the constraint system is set up with completeness so that the proofs generated with the correct set of witnesses indeed pass the ZKP verification.

Missing range checks. This is a popular type of an underconstrained-circuit vulnerability. Due to the usage of field arithmetic, overflow checks and range checks serve a huge purpose to build applications that work over the integers. We manually check the code for such missing checks and, in certain cases, provide a proof that the given set of range checks is sufficient to constrain the circuit up to specification.

Cryptography. ZKP technology and their applications are based on various aspects of cryptography. We manually review the cryptography usage of the project and examine the relevant studies and standards for any inconsistencies or vulnerabilities.

Code maturity. We look for potential improvements in the codebase in general. We look for violations of industry best practices, guidelines, and code quality standards.

For each finding, Zellic assigns it an impact rating based on its severity and likelihood. There is no hard-and-fast formula for calculating a finding's impact. Instead, we assign it on a case-by-case basis based on our judgment and experience. Both the severity and likelihood of an issue affect its impact. For instance, a highly severe issue's impact may be attenuated by a low likelihood.

We assign the following impact ratings (ordered by importance): Critical, High, Medium, Low, and Informational.

Zellic organizes its reports such that the most important findings come first in the document, rather than being strictly ordered on impact alone. Thus, we may sometimes emphasize an "Informational" finding higher than a "Low" finding. The key distinction is that although certain findings may have the same impact rating, their *importance* may differ. This varies based on various soft factors, like our clients' threat models, their business needs, and so on. We aim to provide useful and actionable advice to our partners considering their long-term goals, rather than a simple list of security issues at present.

2.3. Scope

The engagement involved a review of the following targets:

AltDA OP Succinct Integration Circuits

Type	Rust
Platform	SP1zkVM
Target	op-succinct
Repository	https://github.com/succinctlabs/op-succinct
Version	78129507cb0a46a1983606d403eb5b487d8369f9
Programs	utils/altda/** programs/range/altda/** programs/range/utils/** utils/client/src/boot.rs utils/client/src/witness/executor.rs utils/client/src/witness/mod.rs utils/client/src/witness/preimage_store.rs utils/client/src/client.rs utils/client/src/oracle/blob_provider.rs

2.4. Project Overview

Zellic was contracted to perform a security assessment for a total of 1.2 person-weeks. The assessment was conducted by two consultants over the course of four calendar days.

Contact Information

The following project manager was associated with the engagement:

Jacob Goreski
↗ Engagement Manager
jacob@zellic.io ↗

The following consultants were engaged to conduct the assessment:

Ziling Chen
↗ Security Engineer
ziling@zellic.io ↗

Avi Weinstock
↗ Security Engineer
avi@zellic.io ↗

2.5. Project Timeline

The key dates of the engagement are detailed below.

September 18, 2026 Start of primary review period

September 23, 2026 End of primary review period

3. Detailed Findings

3.1. The zkVM does not bind the first digest byte of an AltDA Keccak commitment

Target	utils/altda/client/src/data_source.rs		
ID	ZELIC-ZFP86Q		
Category	Coding Mistakes	Severity	Medium
Likelihood	Medium	Impact	Medium

Description

The AltDA guest passes the Keccak commitment obtained from L1 directly to `PreimageKey::new_keccak256()`, reads the batch data from the preimage oracle under that key, and relies on `PreimageStore::check_preimages()` to verify its integrity:

```
// utils/altda/client/src/data_source.rs
let resolved_data = self
    .oracle
    .get(PreimageKey::new_keccak256(commitment_hash))
    .await
    .map_err(|e| PipelineError::Provider(e.to_string()).temp())?;
```

Kona's `PreimageKey` reserves byte zero for the key type and retains only bytes 1 through 31 of the supplied digest:

```
pub fn new(key: [u8; 32], key_type: PreimageKeyType) -> Self {
    let mut data = [0u8; 31];
    data.copy_from_slice(&key[1..]);
    Self { data, key_type }
}
```

The `PreimageStore::check_preimages()` function recomputes the Keccak digest but converts it to the same truncated key before comparing it with the requested key:

```
// utils/client/src/witness/preimage_store.rs
PreimageKeyType::Keccak256 => Some(keccak256(value).0),

// ...

if key != &PreimageKey::new(expected_hash, key.key_type()) {
    return Err(PreimageOracleError::InvalidPreimageKey);
}
```

```
}
```

Consequently, the integrity check establishes equality for only the final 31 bytes:

```
keccak256(batch_data)[1..32] == commitment_hash[1..32]
```

OP AltDA requires equality across the complete Keccak digest:

```
keccak256(batch_data) == commitment_hash
```

Two Keccak digests that differ only in their first byte therefore map to the same PreimageKey. Successful preimage validation does not establish that the complete Keccak digest of the batch data equals the commitment from L1. The unbound byte is the first byte of the Keccak digest, not the outer derivation version or commitment type byte.

Impact

The zkVM may accept batch data whose complete Keccak digest does not match its L1 commitment.

Recommendations

Compare the complete Keccak digest of the batch data with its L1 commitment inside the guest.

Remediation

This issue has been acknowledged by Succinct Labs, and fixes were implemented in the following commits:

- [3a107674](#) ↗
- [1d2e418c](#) ↗

3.2. The client applies the deposit-only fallback to every execution error

Target	utils/client/src/client.rs	ID	ZELIC-VWP7VZ
Category	Coding Mistakes	Severity	Medium
Likelihood	Low	Impact	Medium

Description

After deriving a payload, `advance_to_target()` calls `execute_payload()`. When Holocene is active, every error returned by this call enters the same branch. The client flushes the current channel, removes all transactions other than deposits, and executes the modified payload:

```
// utils/client/src/client.rs
let outcome = match driver.executor.execute_payload(attributes.clone()).await {
    {
        Ok(outcome) => outcome,
        Err(e) => {
            error!(target: "client", "Failed to execute L2 block: {}", e);

            if cfg.is_holocene_active(attributes.payload_attributes.timestamp) {
                driver.pipeline.signal(Signal::FlushChannel).await?;

                attributes.transactions = attributes.transactions.map(|txs| {
                    txs.into_iter()
                        .filter(|tx| !tx.is_empty() && tx[0]
== OpTxType::Deposit as u8)
                        .collect::<Vec<_>>()
                });

                driver.executor.update_safe_head(tip_cursor.l2_safe_head_header.clone());
                match driver.executor.execute_payload(attributes.clone()).await {
                    Ok(header) => header,
                    Err(e) => return Err(DriverError::Executor(e)),
                }
            } else {
                continue;
            }
        }
    }
};
```

The `execute_payload()` function can fail for multiple reasons, including invalid payload fields or transactions, missing bytecode data, provider failures, and internal execution errors. An error therefore does not necessarily mean that the payload is invalid. The current code does not distinguish these causes before applying the deposit-only fallback.

Impact

The client may derive a different L2 output root after an error unrelated to payload validity.

Recommendations

Apply the deposit-only fallback only to errors that establish payload invalidity.

Remediation

This issue has been acknowledged by Succinct Labs, and fixes were implemented in the following commits:

- [81f09688](#) ↗
- [58649923](#) ↗

3.3. The AltDA host does not limit HTTP response bodies

Target	utils/altda/host/src/handler.rs	ID	ZELIC-DVHJJE
Category	Coding Mistakes	Severity	Low
Likelihood	Medium	Impact	Low

Description

The AltDA host retrieves batch data from the configured DA server. The HTTP client sets a request time-out of 30 seconds but does not limit the response body size. After receiving a successful response, the handler reads the complete body and copies it before storing it in the key-value store:

```
// utils/altda/host/src/handler.rs
let batch_data = response
    .bytes()
    .await
    .map_err(|e| anyhow::anyhow!("Failed to read DA server response body: {e}"))?;

// ...

let mut kv_lock = kv.write().await;
kv_lock.set(
    PreimageKey::new_keccak256(commitment_hash).into(),
    batch_data.to_vec(),
)?;
```

The handler neither rejects an oversized Content - Length before reading the body nor counts the bytes received from a chunked response. The request time-out limits the duration but not the amount of data that may be downloaded and allocated.

The client applies its input size limit only after `resolve_keccak256_commitment()` returns:

```
// utils/altda/client/src/data_source.rs
let result = self.resolve_keccak256_commitment(commitment).await;

// ...

if data.len() as u64 > self.max_input_size {
    return Err(PipelineError::NotEnoughData.temp());
}
```

```
}
```

By this point, the host has downloaded, buffered, copied, and stored the complete response.

Impact

An oversized response may exhaust host memory and interrupt proof generation.

Recommendations

Enforce the configured input size limit while reading each HTTP response.

Remediation

This issue has been acknowledged by Succinct Labs, and they have provided the following response:

A host-only response cap stops witness generation before the guest can verify and skip an oversized committed batch.

4. System Design

This provides a description of the high-level components of the system and how they interact, including details like a function's externally controllable inputs and how an attacker could leverage each input to cause harm or which invariants or constraints of the system are critical and must always be upheld.

Not all components in the audit scope may have been modeled. The absence of a component in this section does not necessarily suggest that it is safe.

4.1. Component: AltDA Client and Host

Description

The AltDA Client verifies L2 execution inside of the SP1 zkVM, producing a proof that is verified on an L1. The range program with the zkVM entry point wraps the client crate's `AltDAWitnessExecutor`, reading the input from the host; passes it to the executor; and commits to its output. The input `DefaultWitnessData` contains a `PreimageStore` mapping keys to their Keccak-256/SHA-256 preimages and `BlobData` containing KZG commitments to binary data. The output is a `BootInfoStruct` containing the old and new L2 state roots; the new L2 block number; an L1 block hash, which commits to the old state root; and a hash of the roll-up configuration.

The `PreimageStore` struct implements Kona's `PreimageOracleClient` trait, and the `BlobData` struct is converted into a `BlobStore` struct that implements Kona's `BlobProvider` trait, which are used to provide data to Kona's executor and driver for computing updated L2 state for the target block number. These are wrapped in an `AltDADataSource` struct, which implements Kona's `DataAvailabilityProvider` trait for Kona's `OraclePipeline`. The `PreimageStore` struct does not validate the entire hash in `check_preimages`; see Finding [3.1](#). The KZG proofs in for the `BlobData` are verified with `kzg_rs` in the process of constructing the `BlobStore`. The `kzg_rs` crate is additionally wrapped as an `revm` precompile to accelerate computing the updated L2 state.

The `BootInfo` containing the initial state is passed in the `PreimageStore` as local preimage keys, which are not hashed. The computed L2 state root and block number are validated against the claimed values.

The AltDA Host is outside the SP1 zkVM and fetches data from AltDA to populate the witness data. It does not limit the lengths of the responses, which can lead to failure to generate a proof — but not an invalid proof (see Finding [3.3](#)).

Test coverage

Cases covered

- Serializing and deserializing the roll-up configuration produces a deterministic hash, for both modified and unmodified configurations.
- The comparison of block numbers is tested for both success and failure cases.
- Verification of KZG proofs is tested for both success and failure cases.
- Various configurations of plumbing data through an `AltDADataSource` are tested.

Cases not covered

- The overall flow of the client is not covered by unit tests but is covered by E2E/integration tests.

Attack surface

For the client, the witness data is considered attacker provided and is validated by the client to ensure soundness. For the host, invalid data provided by upstream servers results in failures of completeness.

5. Assessment Results

During our assessment on the scoped AltDA OP Succinct Integration circuits, we discovered three findings. No critical issues were found. Two findings were of medium impact and one was of low impact.

5.1. Disclaimer

This assessment does not provide any warranties about finding all possible issues within its scope; in other words, the evaluation results do not guarantee the absence of any subsequent issues. Zellic, of course, also cannot make guarantees about any code added to the project after the version reviewed during our assessment. Furthermore, because a single assessment can never be considered comprehensive, we always recommend multiple independent assessments paired with a bug bounty program.

For each finding, Zellic provides a recommended solution. All code samples in these recommendations are intended to convey how an issue may be resolved (i.e., the idea), but they may not be tested or functional code. These recommendations are not exhaustive, and we encourage our partners to consider them as a starting point for further discussion. We are happy to provide additional guidance and advice as needed.

Where Zellic states that a finding has been addressed or fixed in one or more specific commits, this indicates only that those commits introduce changes that, in our assessment, address the finding relative to the codebase version originally reviewed. This does not imply that Zellic reviewed other changes contained in those commits, the overall state of the codebase at those commits, or the interplay between remediation measures for different findings.

Finally, the contents of this assessment report are for informational purposes only; do not construe any information in this report as legal, tax, investment, or financial advice. Nothing contained in this report constitutes a solicitation or endorsement of a project by Zellic.